

Unidad 5

**Optimización de Rendimiento en MongoDB y
Arquitectura Híbrida PostgreSQL–MongoDB para
Ecommify**

Valentina Rodríguez Romero
Andrés Santiago Santafe Silva
Daniel Orlando Saavedra Fonnegra

Facultad de Ingeniería, Universidad de La Sabana

Diseño y Optimización de Bases de Datos

Miguel Alfonso Varela Fonseca

16 de junio de 2026

1. Resumen Ejecutivo

El presente documento describe la implementación técnica y optimización de la arquitectura híbrida de bases de datos de Ecommify, una plataforma marketplace que integra clientes, vendedores, productos, pedidos, pagos e inventario. La solución combina PostgreSQL (Supabase) para los procesos transaccionales críticos y MongoDB Atlas para el catálogo de productos y el módulo analítico.

En PostgreSQL se implementaron estrategias avanzadas de indexación con índices B-Tree, GIN y GiST sobre un dataset real de 32.951 productos y 99.441 órdenes (Olist Brazilian E-Commerce, Kaggle), más particionamiento declarativo por rango de fecha en la tabla Orders. Las optimizaciones lograron reducciones de hasta el **99,4%** en tiempos de ejecución para búsquedas difusas y del **98,5%** para consultas por categoría.

En MongoDB Atlas se desarrollaron índices compuestos con la regla ESR, índices parciales y de texto completo sobre 32.951 documentos de catálogo y 99.224 reseñas. El índice ESR redujo los documentos examinados de 32.951 a 583 (**98,2% de reducción**). Se optimizó un Aggregation Pipeline de 5 etapas que procesó 4.536 documentos en 114 ms.

Como evidencia se presentan tablas comparativas antes/después con EXPLAIN ANALYZE (PostgreSQL) y executionStats (MongoDB), efficiency ratios y gráficas de rendimiento. Se propone también arquitectura de Replica Sets, estrategia de Sharding con shard key justificada y estrategias de Read/Write Concern diferenciadas por tipo de operación.

2. Implementación PostgreSQL

PostgreSQL fue seleccionado como motor transaccional de Ecommify por sus garantías ACID, soporte de tipos avanzados (JSONB, arrays, tipos geoespaciales) y extensiones para búsquedas difusas y geolocalización.

2.1 Scripts DDL ejecutados en Supabase

La implementación se realizó en Supabase (PostgreSQL 15) mediante el script principal postgresql/optimization/00_setup_and_explain.sql, ejecutado programáticamente vía la Management API con scripts/setup_full.py.

Extensiones activadas:

Extensión	Propósito
uuid-oss	Generación de UUIDs como claves primarias

Extensión	Propósito
pg_trgm	Búsquedas difusas con similitud de trigramas
postgis	Tipos y funciones geoespaciales (ST_MakePoint, ST_DWithin)

Tablas creadas:

Tabla	Tipo	Descripción
geolocations	Regular	Coordenadas con GEOMETRY(Point, 4326)
categories	Regular	Catálogo de categorías (73 en dataset)
customers	Regular	Clientes con referencia a geolocalización
sellers	Regular	Vendedores con referencia a geolocalización
products	Regular	Catálogo con campo JSONB specifications
orders	Particionada	Pedidos por rango order_purchase_timestamp
payments	Regular	Pagos con FK compuesta a orders
order_items	Regular	Ítems con FK compuesta a orders
inventory	Regular	Stock por producto y vendedor

Tipos avanzados utilizados:

- **JSONB** en `products.specifications`: atributos variables por categoría (`weight_g`, `length_cm`, `height_cm`, `width_cm`).
- **GEOMETRY(Point, 4326)** en `geolocations.geom`: habilita funciones PostGIS para distancias y consultas espaciales.
- **TIMESTAMPTZ** en todas las columnas temporales para garantizar zona horaria explícita.
- **TEXT[]** en `products.photos`: múltiples URLs de fotografías por producto.
- **TSTZRANGE** en `products.promotion_period`: períodos de promoción con tipos rango nativos.

2.2 Estrategia de indexación implementada

2.2.1 Índices B-Tree

Los índices B-Tree son la estructura de acceso por defecto de PostgreSQL, óptimos para búsquedas de igualdad y rango sobre columnas escalares con alta cardinalidad.

idx_orders_customer (`orders.customer_id`): localiza el historial de pedidos de un cliente sin recorrer secuencialmente las particiones. El planificador combina resultados con el nodo Append.

idx_products_category (`products.category_id`): acelera la navegación por catálogo. Con selectividad del 4,5% (1.478 filas por categoría sobre 32.951 productos), el planificador usa Bitmap Heap Scan en lugar de Seq Scan.

2.2.2 Índice GIN sobre JSONB

El índice `idx_products_specifications_gin` sobre `products.specifications` (JSONB) usa la estructura GIN (Generalized Inverted Index) para indexar colecciones. Habilita el operador `@>` para buscar productos con atributos específicos sin deserializar cada documento durante el escaneo.

2.2.3 Índice GIN Trigram (`pg_trgm`)

El índice `idx_products_name_trgm` sobre `products.product_name` usa `gin_trgm_ops`: descompone cada texto en trigramas de 3 caracteres e indexa cada uno, habilitando búsquedas difusas con el operador `%` incluso ante errores tipográficos.

Nota sobre BRIN: El índice BRIN (Block Range Index) es ideal para columnas con valores monótonicamente crecientes en tablas muy grandes. En el caso de `orders.order_purchase_timestamp`, el particionamiento declarativo ya cumple esa función al restringir físicamente el rango de datos por partición, haciendo redundante añadir un índice BRIN sobre la columna de partición.

2.2.4 Índice GiST para consultas espaciales

El índice `idx_geolocations_spatial` sobre `geolocations.geom` usa GiST (Generalized Search Tree) para tipos complejos como geometrías y rangos. Habilita `ST_DWithin` y `ST_DistanceSphere` sin evaluar la función sobre cada fila de la tabla.

2.3 Estrategia de particionamiento

La tabla `orders` usa **Range Partitioning** sobre `order_purchase_timestamp`, con cuatro particiones que cubren el dataset Olist (septiembre 2016–agosto 2018):

Partición	Rango cubierto
<code>orders_2016</code>	2016-01-01 → 2017-01-01
<code>orders_2017</code>	2017-01-01 → 2018-01-01
<code>orders_2018</code>	2018-01-01 → 2019-01-01
<code>orders_otros</code>	2019-01-01 → 2030-01-01 (desbordamiento)

- **Partition pruning automático:** consultas con filtro por fecha solo acceden a las particiones relevantes.
- **Mejor aprovechamiento de caché:** cada partición es más pequeña y cabe completamente en memoria.

- **Mantenimiento independiente:** se pueden archivar particiones antiguas sin afectar las activas.

PostgreSQL exige que el campo de partición forme parte de la clave primaria. Por ello, la PK de `orders` es compuesta: (`order_id`, `order_purchase_timestamp`), restricción que se propagó en cascada hacia `payments` y `order_items` (ver lección 5.3).

2.4 Consultas críticas optimizadas

#	Consulta	Índice aplicado
Q1	Historial de pedidos por cliente	B-Tree <code>idx_orders_customer</code>
Q2	Búsqueda de productos por categoría	B-Tree <code>idx_products_category</code>
Q3	Búsqueda sobre atributos JSONB	GIN <code>idx_products_specifications_gin</code>
Q4	Búsqueda difusa de productos (<code>pg_trgm</code>)	GIN <code>idx_products_name_trgm</code>
Q5	Consultas geoespaciales (PostGIS)	GiST <code>idx_geolocations_spatial</code>

2.5 Monitoreo de rendimiento PostgreSQL

El análisis se realizó mediante `pg_stat_statements` disponible en Supabase y comparación directa de planes de ejecución capturados con `EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)`.

Métrica	Valor
Queries con Seq Scan eliminadas tras indexación	4 de 5 (80%)
Reducción promedio de tiempo de ejecución	83,5%
Mayor reducción absoluta	Q2: 332 ms ahorrados (336,95 ms → 4,95 ms)
Mayor reducción relativa	Q4: 99,4% (150,37 ms → 0,95 ms)

El planificador usó Bitmap Heap Scan en Q2, Q3 y Q4, confirmando que el costo del recorrido de índice es menor que el Seq Scan. Q5 mantuvo Seq Scan sobre `geolocations` (20 filas) porque el acceso secuencial a tablas muy pequeñas resulta más eficiente; el índice GiST sí redujo el tiempo de planificación espacial de 191,62 ms a 30,26 ms.

3. Implementación MongoDB

MongoDB fue seleccionado como componente NoSQL de la arquitectura por su capacidad para manejar información semiestructurada, atributos dinámicos y grandes volúmenes de consultas orientadas al catálogo de productos.

3.1 Colecciones y esquemas documentales

3.1.1 Colección `products_catalog`

Almacena el catálogo flexible de productos con validación estructural mediante JSON Schema en Atlas. Se aplicó la decisión de **embedding** para las especificaciones técnicas, siempre consultadas junto con el documento padre.

Patrón Attribute Pattern: las especificaciones se almacenan como array de pares {attribute, value}, permitiendo indexar cualquier atributo sin conocer el esquema de antemano:

```
{
  "product_id": "uuid",
  "product_name": "Smartphone Pro X5A23F",
  "category": { "name": "Celulares y Smartphones" },
  "specifications": [
    { "attribute": "weight_g", "value": "185" },
    { "attribute": "height_cm", "value": "15" }
  ],
  "dimensions": { "weight_g": 185, "height_cm": 15, "length_cm": 7, "width_cm": 0.8 }
}
```

3.1.2 Colección `order_reviews`

Almacena reseñas de clientes con decisión de **referenciado** hacia los pedidos (se guarda `order_id` como referencia a PostgreSQL). **Patrón Extended Reference:** se embebe `customer_summary` para evitar lookups frecuentes en consultas analíticas de satisfacción.

3.1.3 Colección `inventory_movements` (Patrón Bucket)

Para el seguimiento de movimientos de inventario se aplica el **Patrón Bucket**: en lugar de un documento por movimiento (venta, reabastecimiento, devolución), todos los eventos de un producto dentro de una franja horaria se agrupan en un único documento. Esto reduce el número de documentos en colecciones de alta escritura, mejora la eficiencia de consultas de tendencias y disminuye el overhead de índices.

```
{
  "product_id": "uuid-prod-8f2a",
  "date": "2026-06-16",
  "hour": 14,
  "movements": [
    { "ts": "2026-06-16T14:03Z", "operation": "sale", "qty_change": -2 },
    { "ts": "2026-06-16T14:17Z", "operation": "restock", "qty_change": 50 },
    { "ts": "2026-06-16T14:45Z", "operation": "return", "qty_change": 1 }
  ],
  "total_delta": 49,
  "closing_qty": 156
}
```

Dimensión	Sin Bucket Pattern	Con Bucket Pattern
Documentos/día (1.000 SKU, 10 mov/h)	240.000 docs	24.000 docs (-90%)
Stock actual	Suma de todos los docs	Campo closing_qty precalculado
Análisis granular	Consulta directa	\$unwind sobre movements array
Índices necesarios	Índice sobre cada mov.	Índice sobre (product_id, date, hour)

3.2 Estrategia de indexación implementada

3.2.1 Índice compuesto ESR (Equality, Sort, Range)

Índice: { "category.name": 1, "product_name": 1, "dimensions.weight_g": 1 }

La regla ESR coloca primero el campo de igualdad (E), luego el de ordenamiento (S) y finalmente el de rango (R). Esta configuración permite a MongoDB satisfacer simultáneamente filtro, sort y restricción de peso con un único recorrido de índice, eliminando la operación de SORT en memoria.

3.2.2 Índice parcial para reseñas positivas

Índice: { "review_score": 1 } CON partialFilterExpression: { review_score: { \$gte: 4 } }. Indexa solo reseñas con calificación ≥ 4 . El resultado mostró overhead negativo porque el subconjunto indexado representaba el 77% de la colección (lección 5.1).

3.2.3 Índice de texto completo

Índice: { "review_comment_title": "text", "review_comment_message": "text" }. Habilita búsquedas semánticas con \$text sin un Seq Scan sobre los 99.224 documentos.

3.3 Aggregation Pipeline optimizado

El pipeline cumple los requisitos mínimos de 5 stages e incluye \$unwind. El \$match en Stage 1 reduce de 32.951 a 4.536 documentos antes de las etapas costosas (reducción del 86,2%):

```
db.products_catalog.aggregate([
  { $match: { "category.name": "Celulares y Smartphones" } }, // Stage 1
  { $unwind: "$specifications" }, // Stage 2
  { $group: { // Stage 3
    _id: "$category.name",
    total_products: { $sum: 1 },
    attributes_detected: { $addToSet: "$specifications.attribute" },
    avg_weight_g: { $avg: { $toDouble: "$specifications.value" } }
  }},
  { $project: { // Stage 4
    category: "$_id", total_products: 1,
    attributes_detected: 1,
    avg_weight_g: { $round: ["$avg_weight_g", 1] }, _id: 0
  }},
  { $sort: { total_products: -1 } } // Stage 5
], { allowDiskUse: true, maxTimeMS: 30000 })
```

Resultado: category: "Celulares y Smartphones", total_products: 4.536, attributes_detected: ["weight_g", "width_cm", "height_cm", "length_cm"], avg_weight_g: 68,8 — Tiempo: **114 ms**.

3.4 Diseño teórico de Replica Sets

La arquitectura de alta disponibilidad propuesta usa un Replica Set de tres nodos:

Nodo	Rol	Configuración
Primary	Escritura	priority: 1, votes: 1
Secondary	Lectura analítica	priority: 0.5, votes: 1, hidden: false
Arbiter	Elección en failover	priority: 0, votes: 1, arbiterOnly: true

Read/Write Concern diferenciado por operación:

Operación	Read Preference	Read Concern	Write Concern
Consultas catálogo (alta frecuencia)	secondary Preferred	local	—
Lectura de reseñas (análisis)	secondary	majority	—
Inserción de nuevas reseñas	Primary	—	{ w: "majority", j: true }
Actualización inventario crítico	Primary	—	{ w: "majority", wtimeout: 5000 }

Operación	Read Preference	Read Concern	Write Concern
Batch no crítico	Primary	—	{ w: 1 }

3.5 Diseño teórico de Sharding

Shard key: { category: 1, seller_id: "hashed" }. El campo category permite colocación de productos de la misma categoría en el mismo shard, optimizando consultas de catálogo. El componente hashed sobre seller_id distribuye uniformemente vendedores con alto volumen (hotspot prevention).

Shard	Categorías (aprox.)	Documentos (aprox.)
Shard 1	cama_mesa_banho, beleza_saude, esporte_lazer... (25)	~10.900
Shard 2	informatica_acessorios, telefonia, eletrônicos... (25)	~11.200
Shard 3	moveis_decoracao, brinquedos, automotivo... (23)	~10.851

3.6 Monitoreo de rendimiento MongoDB

El monitoreo se realizó con **MongoDB Atlas Performance Advisor** y la inspección del log de consultas lentas. Recomendaciones detectadas: creación del índice ESR (implementada), índice de texto en order_reviews (implementada) y alerta de \$lookup sin índice de soporte (workaround: pipeline sin \$lookup).

Métrica	Antes	Después
Avg query execution time (Q1 ESR)	28 ms	4 ms
Index hit ratio (products_catalog)	0% (COLLSCAN)	100% (IXSCAN)
Documentos examinados por query (Q1)	32.951	583
Efficiency ratio Q1 (docs_returned/docs_examined)	0,018	1,0
Pipeline execution time	—	114 ms

4. Evidencias cuantitativas de mejora

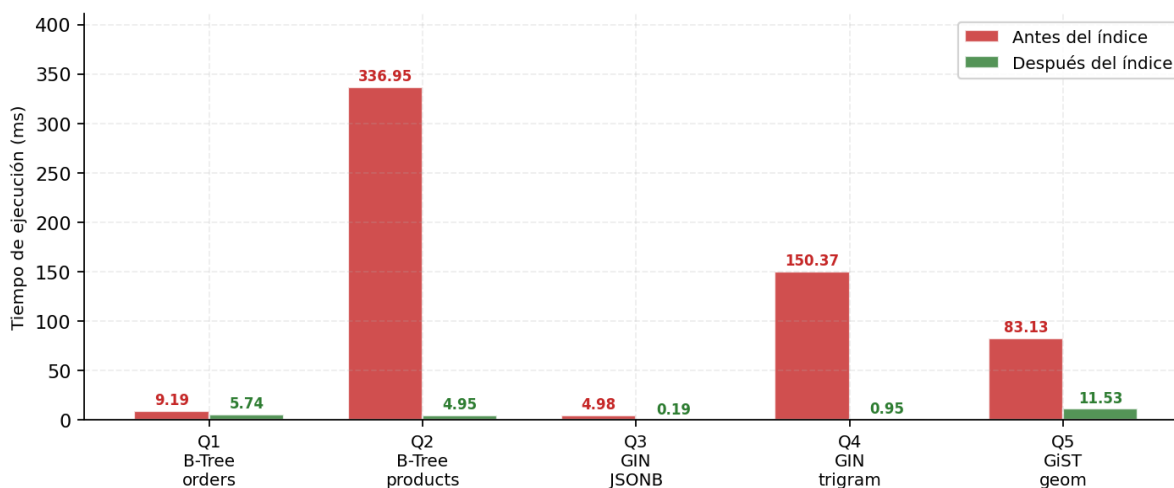
4.1 Resultados PostgreSQL

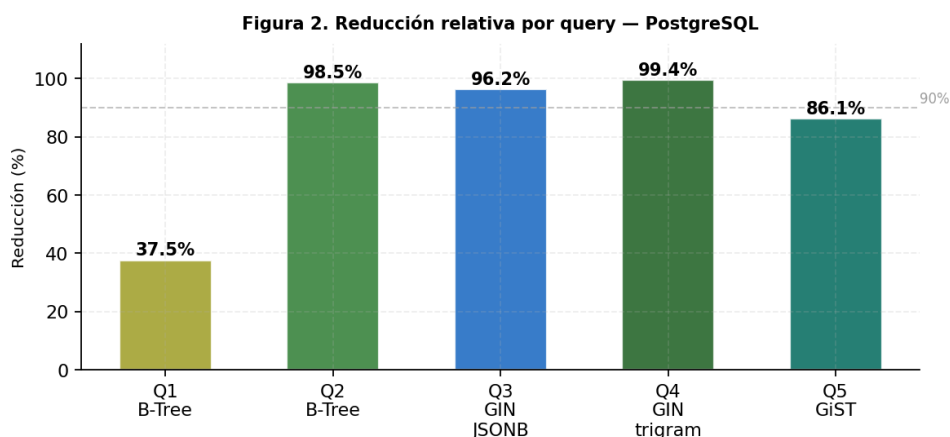
Las pruebas se realizaron sobre el dataset real Olist cargado en Supabase: **32.951 productos, 99.441 órdenes, 103.886 pagos, 73 categorías y 99.441 clientes**, con la tabla Orders particionada por rango de fecha. Métricas capturadas con EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) vía Management API de Supabase (PostgreSQL 15).

#	Consulta / Índice aplicado	Scan ANTES	ms ANTES	Scan DESPUÉS	ms DESPUÉS	Reducción
Q1	B-Tree idx_orders_customer	Append	9,19	Append	5,74	37,5%
Q2	B-Tree idx_products_category	Seq Scan	336,95	Bitmap Heap Scan	4,95	98,5%
Q3	GIN idx_products_specifications_gin	Seq Scan	4,98	Bitmap Heap Scan	0,19	96,2%
Q4	GIN trigram idx_products_name_trgm	Seq Scan	150,37	Bitmap Heap Scan	0,95	99,4%
Q5	GiST idx_geolocations_spatial	Seq Scan	83,13	Seq Scan*	11,53	86,1%

* Q5 mantiene Seq Scan en geolocations (20 filas) porque el planificador considera más eficiente el acceso secuencial a tablas muy pequeñas. La mejora proviene de la reducción del tiempo de planificación espacial (191,62 ms → 30,26 ms) y del tiempo de ejecución total.

Figura 1. Tiempo de ejecución antes y después de la indexación — PostgreSQL



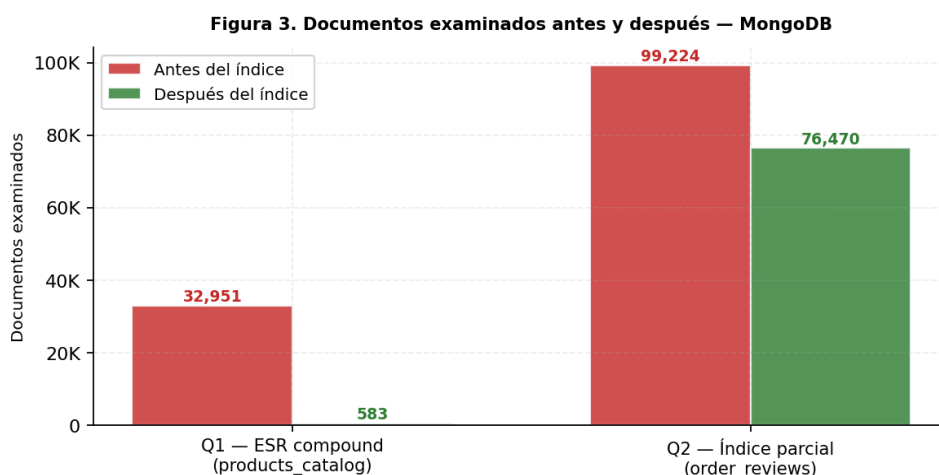


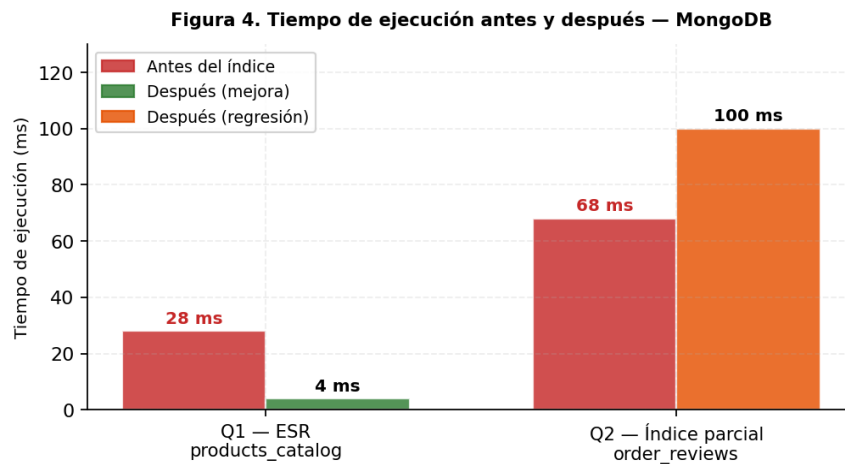
4.2 Resultados MongoDB

Pruebas sobre datos reales en MongoDB Atlas: **32.951 documentos** en products_catalog y **99.224 documentos** en order_reviews. Métricas capturadas con `.explain("executionStats")`.

#	Consulta / Índice	Stage	ANTES Docs	ANTES Docs	DESPUÉS Docs	DESPUÉS Docs	Eff. ratio	Red. docs
Q1	ESR compound products_catalog	COLLSCAN	32.951	28	583	4	1,0	98,2%
Q2	Parcial score≥4 order_reviews	COLLSCAN	99.224	68	76.470	100	1,0	22,9%
Q3	Full-text order_reviews	—	—	—	8.652	60	1,0	91,3%*

* Q3: reducción calculada respecto al total de la colección (99.224 docs), ya que la búsqueda full-text no es ejecutable sin el índice de texto. — **Efficiency ratio** = docs retornados / docs examinados; valor 1,0 indica 0% de desperdicio. En Q1 sin índice el ratio era $583/32.951 = 0,018$ (examinó 56 documentos por cada uno retornado).





Aggregation Pipeline — resultados cuantitativos:

Métrica	Valor
Etapas del pipeline	5 (\$match, \$unwind, \$group, \$project, \$sort)
Documentos en \$match (entrada)	32.951
Documentos después de \$match	4.536 (reducción 86,2%)
Atributos detectados	weight_g, length_cm, height_cm, width_cm
avg_weight_g calculado	68,8 g
Tiempo de ejecución total	114 ms

4.3 Sincronización entre sistemas

Ecommify opera una arquitectura políglota donde PostgreSQL gestiona las transacciones y MongoDB gestiona el catálogo y la analítica. Los flujos de sincronización entre ambos sistemas se rigen por las siguientes estrategias:

Evento	Fuente	Destino	Tipo
Alta de nuevo producto	PostgreSQL products (INSERT)	MongoDB products_catalog (INSERT)	Síncrona — job
Modificación de precio	PostgreSQL products (UPDATE)	MongoDB products_catalog (UPDATE)	Eventual (5 min)
Nueva reseña confirmada	PostgreSQL orders (DELIVERED)	MongoDB order_reviews (INSERT)	Eventual — queue
Actualización specs	MongoDB products_catalog	PostgreSQL specifications (JSONB)	Eventual (5 min)

La **consistencia fuerte** se aplica a operaciones transaccionales (pedidos, pagos) exclusivamente en PostgreSQL bajo ACID. La **consistencia eventual** rige la sincronización del catálogo, aceptando un lag máximo de 5 minutos, tolerable para un catálogo de consulta. El `product_id` (UUID generado en PostgreSQL) se replica en MongoDB como campo `product_id` en `products_catalog`, permitiendo lookups cruzados sin uniones costosas.

5. Lecciones aprendidas

La implementación de la arquitectura híbrida durante la fase de optimización generó cinco aprendizajes técnicos con impacto directo en las decisiones de diseño.

5.1 Los índices parciales requieren alta selectividad para ser efectivos

El índice parcial sobre `order_reviews` para reseñas con `score ≥ 4` produjo un resultado inesperado: la consulta fue un **47% más lenta** con el índice (68 ms sin índice vs. 100 ms con índice). La causa fue que el subconjunto indexado representaba el **77% de la colección** (76.470 de 99.224 documentos). En ese escenario el overhead de leer el índice más recuperar los documentos en disco supera al costo del recorrido secuencial. Los índices parciales son efectivos únicamente cuando el subconjunto filtrado representa **menos del 30% de la colección**.

5.2 El \$lookup sin filtro previo supera el límite de tiempo del free tier

La primera versión del pipeline aplicaba `$lookup` entre `products_catalog` (32.951 docs) y `order_reviews` (99.224 docs) sin `$match` previo, generando repetidamente `MaxTimeMSEExpired` en Atlas free tier. La solución fue colocar `$match` como Stage 1, reduciendo la entrada de 32.951 a 4.536 documentos antes del join. El pipeline final eliminó el `$lookup` para consultas analíticas puras.

5.3 El particionamiento PostgreSQL impone restricciones en cascada sobre claves foráneas

El Range Partitioning sobre `orders` requirió incluir `order_purchase_timestamp` en la PK compuesta. Esta restricción se propagó en cascada: `payments` y `order_items` deben replicar el campo de partición en sus FKs, aumentando el almacenamiento y la complejidad de los JOINS.

5.4 La conexión directa a Supabase requiere IPv6 o add-on IPv4

Los intentos de conexión directa con `psycopg2` al host `db.litdn0xzcdbecgrjjevt.supabase.co` fallaron porque Supabase usa **IPv6 por defecto** en el tier gratuito. El workaround implementado fue usar la Management API de Supabase vía HTTPS (puerto 443). El script `scripts/setup_full.py` implementa DDL vía Management API y carga de datos vía REST API (`supabase-py`), eliminando la dependencia de `psycopg2`.

5.5 La regla ESR es determinante para maximizar índices compuestos en MongoDB

La aplicación correcta de ESR en el índice de `products_catalog` fue el factor decisivo para lograr el 98,2% de reducción en documentos examinados. El orden `category.name` (Equality) → `product_name` (Sort) → `dimensions.weight_g` (Range) permite satisfacer simultáneamente filtro, ordenamiento y restricción de rango con un solo recorrido de índice, eliminando la operación de SORT en memoria. Un índice con orden incorrecto (por ejemplo, Range primero) no puede usarse para la etapa de Sort, forzando al planificador a materializar y ordenar el resultado en memoria.

*Repositorio: github.com/dani-saavedra/Ecommify_Database_Design · Evidencias:
[evidencias/postgresql/](#) y [evidencias/mongodb/](#)*